

INGENIERÍA EN ENTORNOS VIRTUALES Y NEGOCIOS DIGITALES

Materia: Programación para Entornos Virtuales

Profesor(a): Ing. Anastasio Rodríguez García

Cursos NetAcad

Fundamentos de python 1

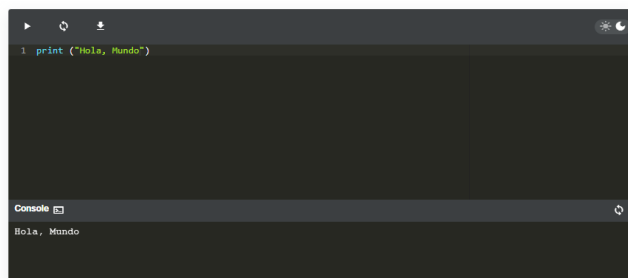
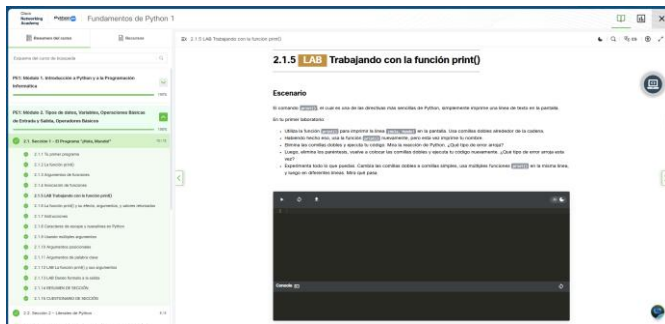
Grupo: GIEV0351

Alumno:

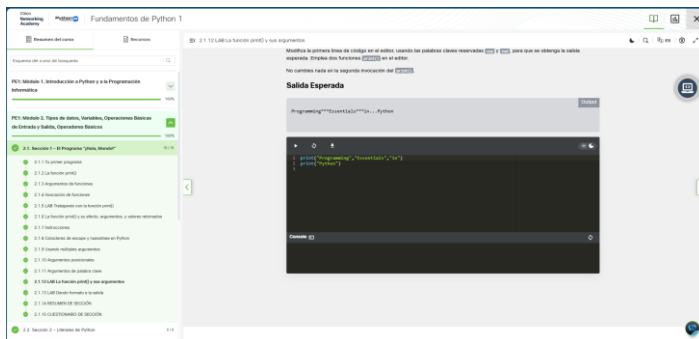
Aguillon Valdez Luis Angel

Fundamentos de Python 1

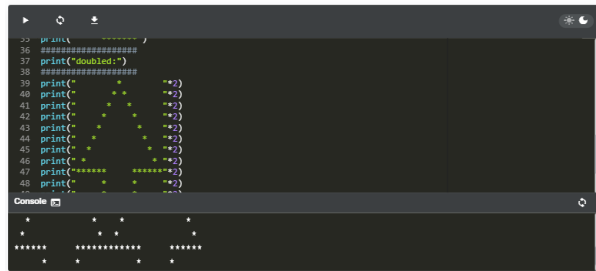
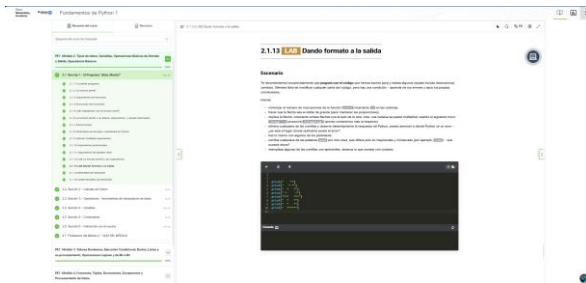
Lab 1



Lab 2

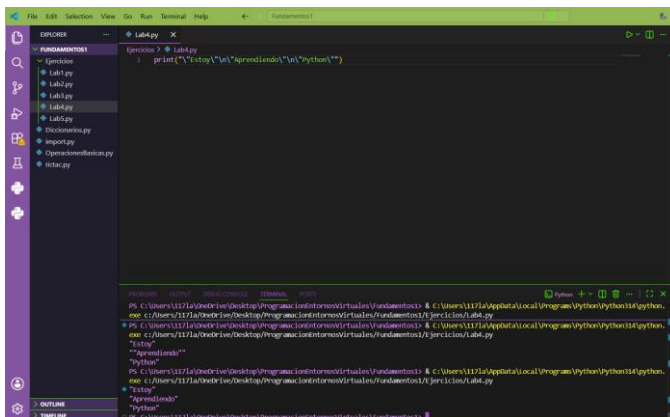


Lab 3

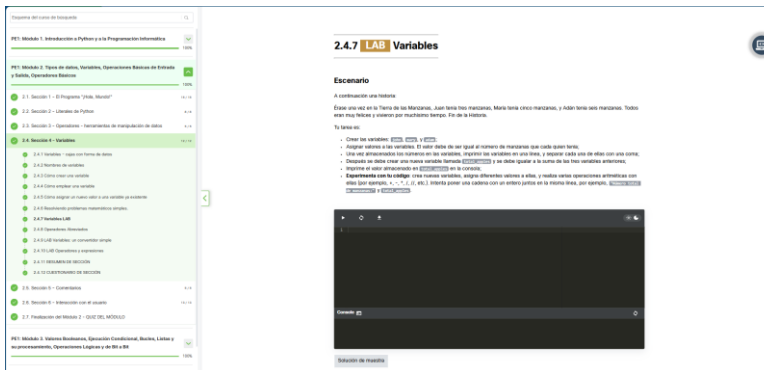


Solución de muestra

Lab 4



Lab 5



2.4.7 LAB Variables

Escenario

A continuación una historia:

Érase una vez en la Tierra de las Manzanas, Juan tenía tres manzanas, María tenía cinco manzanas, y Adán tenía seis manzanas. Todos eran muy felices y vivieron por muchísimo tiempo. Fin de la Historia.

Tu tarea es:

- Crear las variables: `juan`, `maria`, y `adan`.
- Asignar valores a las variables. El valor debe de ser igual al número de manzanas que cada quien tenía.
- Una vez almacenados los números en las variables, imprimir las variables en una línea, y separar cada una de ellas con una coma.
- Después se debe crear una nueva variable llamada `total_manzanas` y se debe igualar a la suma de las tres variables anteriores.
- Imprimir el valor almacenado en `total_manzanas` en la consola.
- Experimenta con tu código: crea nuevas variables, asigna diferentes valores a ellas, y realiza varias operaciones aritméticas con ellas (por ejemplo, `+`, `-`, `*`, `/`, `%`, etc.). Intenta poner una cadena con un entero juntos en la misma línea, por ejemplo, `manzanas_juan` y `total_manzanas`.

```
1 juan = 3
2 maria = 5
3 adan = 6
4
5 print(juan, maria, adan, sep=',')
6
7 total_manzanas = juan + maria + adan
8 print(total_manzanas)
```

Console

```
3,5,6
14
```

Lab 6

Escenario

Millas y kilómetros son unidades de longitud o distancia.

Teniendo en mente que 1 milla equivale aproximadamente a 1.61 kilómetros, complementa el programa en el editor para que convierta de:

- Millas a kilómetros.
- Kilómetros a millas.

No se debe cambiar el código existente. Escribe tu código en los lugares indicados con `###`. Prueba tu programa con los datos que han sido provistos en el código fuente.

Pon mucha atención a lo que está ocurriendo dentro de la función `convert`. Analiza como es que se proveen múltiples argumentos para la función, y como es que se muestra el resultado.

Nota que algunos de los argumentos dentro de la función `convert` son cadenas (por ejemplo, `millas_a_km`), y otros son variables (por ejemplo, `millas`).

Concepto

Hay una cosa interesante más que está ocurriendo. ¿Puedes ver otra función dentro de la función `convert`? Es la función `round`. Su trabajo es redondear la salida del resultado al número de decimales especificados en el paréntesis, y regresar un valor flotante (dentro de la función `convert` se puede encontrar el nombre de la variable, el nombre, una coma, y el número de decimales que se desean mostrar). Se hablará más de esta función muy pronto, no te preocupes si no todo queda muy claro. Solo se quiere impulsar tu curiosidad.

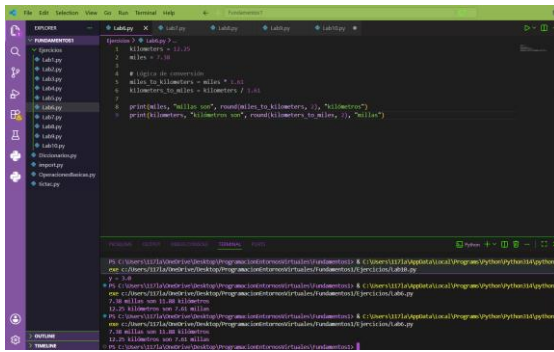
Después de completar el laboratorio, abre Sandbox, y experimenta más. Intenta escribir diferentes convertidores, por ejemplo, un convertidor de USD a EUR, un convertidor de temperatura, etc. (Dijo que tu imaginación vuele! Intenta mostrar los resultados combinando cadenas y variables. Intenta utilizar y experimentar con la función `round` para redondear tus resultados a uno, dos o tres decimales. Revisa que es lo que sucede si no se provee un dígito al redondear. Recuerda probar tus programas.

Experimenta, saca tus propias conclusiones, y aprende. Sé curioso.

Salida Esperada

```
7.38 millas son 11.88 kilometros
12.25 kilometros son 7.61 millas
```

Output



Lab 7

2.4.10 LAB Operadores y expresiones

Escenario

Observa el código en el editor: lee un valor `1000`, lo coloca en una variable llamada `x`, e imprime el valor de la variable llamada `y`. Tu tarea es completar el código para evaluar la siguiente expresión:

$$3x^3 - 2x^2 + 3x - 1$$

El resultado debe ser asignado a `y`.

Recuerda que la notación algebraica clásica muy seguido omite el operador de multiplicación, aquí se debe de incluir de manera explícita. Nota como se cambia el tipo de dato para asegurarnos de que `y` es del tipo `float`.

Mantén tu código limpio y legible, y pruébalo utilizando los datos que han sido proporcionados. No te desanimes por no lograrlo en el primer intento. Se persistente y curioso.

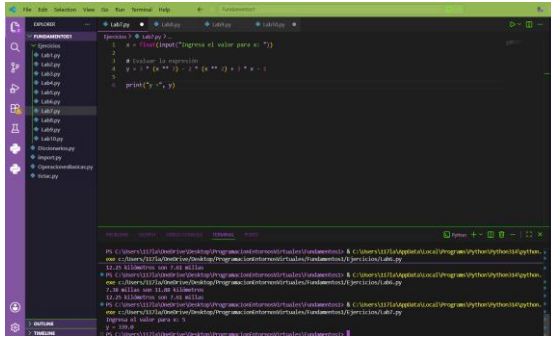
Salida de muestra

x = 0
x = 1
x = -1

Salida de muestra

y = -1.0
y = 3.0
y = -9.0

Output



Lab 8

2.5.3 LAB Comentarios

Escenario

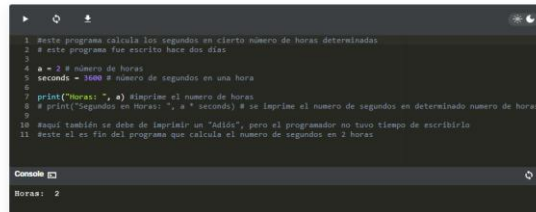
El código en el editor contiene comentarios. Intenta mejorarlo: agrega o quita comentarios donde consideres que sea apropiado (en ocasiones el remover un comentario lo hace mas legible), y cambia el nombre de las variables donde consideres que esto mejorará la comprensión del código.

Nota

Los comentarios son muy importantes. No solo hacen que el programa sea **más fácil de entender**, pero también sirven para **deshabilitar aquellas partes de código que no son necesarias** (por ejemplo, cuando se necesita probar cierta parte del código, e ignorar el resto). Los buenos programadores **describen** cada parte importante del código, y dan **nombres significativos** a variables, debido a que en ocasiones es mucho más sencillo dejar el comentario dentro del código mismo.

Es bueno utilizar nombres de variables **legibles**, y en ocasiones es mejor **dividir el código** en partes con nombres (por ejemplo en funciones). En algunas situaciones, es una buena idea escribir los pasos de como se realizaron los cálculos de una forma sencilla y clara.

Una cosa mas: puede ocurrir que un comentario contenga una pieza de información incorrecta o errónea - ¡nunca se debe de hacer eso a propósito!



Lab 9

2.6.9 LAB Entradas y salidas simples

Escenario

Tu tarea es completar el código para evaluar los resultados de cuatro operaciones aritméticas básicas.

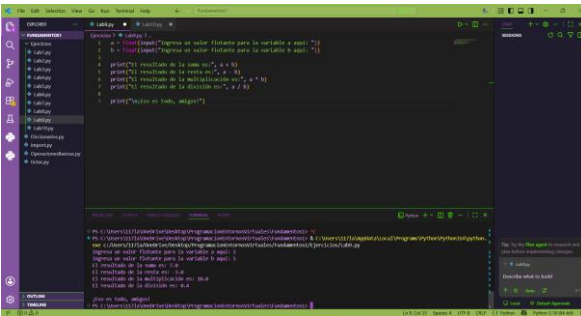
Los resultados deben imprimirse en la consola.

Es posible que no puedas proteger el código de un usuario que quiera dividir entre cero. Está bien, no te preocupes por eso por ahora.

Prueba tu código - ¿produce los resultados esperados?

No te mostraremos ningún dato de prueba - eso sería demasiado simple.

```
1 # Ingresa un valor flotante para la variable a aquí
2 # Ingresa un valor flotante para la variable b aquí
3
4 # Muestra el resultado de la suma aquí
5 # Muestra el resultado de la resta aquí
6 # Muestra el resultado de la multiplicación aquí
7 # Muestra el resultado de la división aquí
8
9 print("\n¡Eso es todo, amigos!")
10
11
```



Lab 10

2.6.10 LAB Operadores y expresiones

Recordatorio

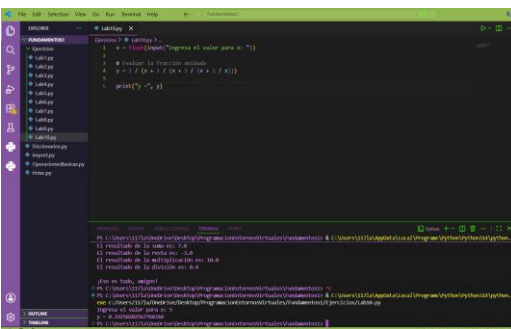
Trabaja con operadores y expresiones para crear expresiones matemáticas.

$$X + \frac{1}{X + \frac{1}{X + \frac{1}{X}}}$$

Problema de matemáticas en lenguaje C. En este problema, debes calcular el valor de la expresión matemática mostrada. Usa variables, operadores y expresiones.

Datos de Prueba

Prueba de entrada	Prueba de salida
1	1.5
2	1.5
3	1.5
4	1.5
5	1.5



Lab 11

2.6.11 LAB Operadores y expresiones – 2

Escenario

La tarea es escribir un código simple para evaluar si encontrar al **dragón Real** de un período de tiempo dado, expresándolo en horas y minutos. La hora de inicio es de cuatro en la tarde (12:00) minutos (0:00). El resultado debe ser verdadero o no, como se muestra.

Por ejemplo, si el evento comienza a las 18:17 y dura 50 minutos, termina a las 19:06.

No se preocupen si su código no es perfecto – está bien si acepta una hora inválida – lo más importante es que el código produzca una salida correcta acorde a la entrada dada.

Problema de código colaborativo. Puede utilizar el operador `input` para leer datos de la entrada.

Datos de Prueba

Entrada de muestra:	Salida Esperada:
12 17 50	19:06 Correct
23 18 40	23:58 Correct
4 1 2000	1:01 Correct

```
def main():
    # Entrada de muestra
    hora_inicio = input()
    minutos_inicio = input()
    duracion = input()

    # Convertir hora y minutos a minutos totales
    minutos_inicio = int(hora_inicio) * 60 + int(minutos_inicio)
    minutos_fin = minutos_inicio + int(duracion)

    # Calcular la hora final (si excede de 60 minutos, restar 60)
    hora_fin = minutos_fin // 60
    minutos_fin = minutos_fin % 60

    # Formatear la hora final
    hora_fin = str(hora_fin).zfill(2)
    minutos_fin = str(minutos_fin).zfill(2)

    # Imprimir el resultado
    print(hora_fin + ":" + minutos_fin)

if __name__ == '__main__':
    main()
```

Lab 12

```
def main():
    # Entrada de muestra
    n = input()

    # Calcular la suma de los cuadrados de los números del 1 al n
    suma = 0
    for i in range(1, n+1):
        suma += i**2

    # Imprimir el resultado
    print(suma)

if __name__ == '__main__':
    main()
```

Lab 13

```
def main():
    # Entrada de muestra
    word_without_vowels = ""

    # Leer la palabra y convertirla a mayúsculas
    word_upper = input("Ingrese una palabra: ")
    word_upper = word_upper.upper()

    # Iterar sobre cada letra de la palabra
    for letter in word_upper:
        # Verificar si la letra es una vocal
        if letter in "AEIOU":
            # Continuar la iteración de la siguiente letra
            continue
        # Concatenar la letra a la palabra sin vocales
        word_without_vowels += letter

    # Imprimir la palabra sin vocales
    print(word_without_vowels)

if __name__ == '__main__':
    main()
```

Lab 14

3.2.14 LAB Fundamentos del bucle while

Escenario

Habrá una vez un sombrero. El sombrero no contiene coraje, sino una lista de cinco números: [1, 2, 3, 4, 5].

Tu tarea es:

- escribir una línea de código que solicite al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1)
- escribir una línea de código que elimine el último elemento de la lista (Paso 2)
- escribir una línea de código que imprima la longitud de la lista existente (Paso 3).

¿Listo para este desafío?



El usuario debe escribir un programa que solicite al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1). El usuario debe escribir una línea de código que elimine el último elemento de la lista (Paso 2). El usuario debe escribir una línea de código que imprima la longitud de la lista existente (Paso 3).

Datos de Prueba

Entrada de usuario	Salida esperada
1	La altura de la pirámide es: 1
2	La altura de la pirámide es: 2
3	La altura de la pirámide es: 3
4	La altura de la pirámide es: 4
5	La altura de la pirámide es: 5

```
def main():
    # Solicita al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1)
    # Elimina el último elemento de la lista (Paso 2)
    # Imprime la longitud de la lista existente (Paso 3)
    # Lista para este desafío?
    hat_list = [1, 2, 3, 4, 5]
    # Paso 1: Solicita al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario
    # Paso 2: Elimina el último elemento de la lista
    # Paso 3: Imprime la longitud de la lista
    print(hat_list)
```

Lab 15

3.4.6 LAB Los fundamentos de las listas

Escenario

Habrá una vez un sombrero. El sombrero no contiene coraje, sino una lista de cinco números: [1, 2, 3, 4, 5].

Tu tarea es:

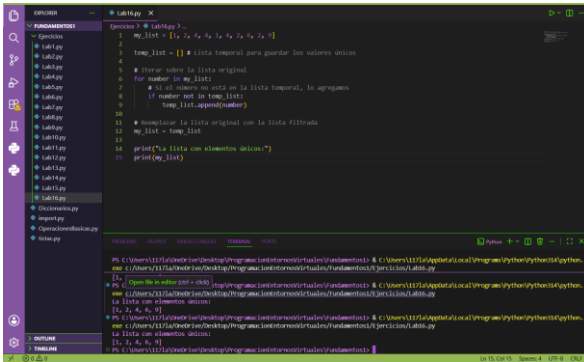
- escribir una línea de código que solicite al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1)
- escribir una línea de código que elimine el último elemento de la lista (Paso 2)
- escribir una línea de código que imprima la longitud de la lista existente (Paso 3).

¿Listo para este desafío?

```
def main():
    # Solicita al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1)
    # Elimina el último elemento de la lista (Paso 2)
    # Imprime la longitud de la lista existente (Paso 3)
    # Lista para este desafío?
    hat_list = [1, 2, 3, 4, 5]
    # Paso 1: Solicita al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario
    # Paso 2: Elimina el último elemento de la lista
    # Paso 3: Imprime la longitud de la lista
    print(hat_list)
```

```
def main():
    # Solicita al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario (Paso 1)
    # Elimina el último elemento de la lista (Paso 2)
    # Imprime la longitud de la lista existente (Paso 3)
    # Lista para este desafío?
    hat_list = [1, 2, 3, 4, 5]
    # Paso 1: Solicita al usuario que reemplace el número central en la lista con un número entero ingresado por el usuario
    # Paso 2: Elimina el último elemento de la lista
    # Paso 3: Imprime la longitud de la lista
    print(hat_list)
```

Lab 16



Lab 17

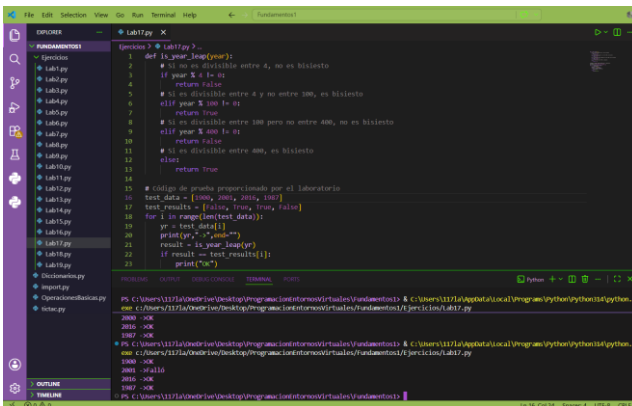
4.3.4 LAB Un año bisiesto: escribiendo tus propias funciones

Tu tarea es escribir y probar una función que toma un argumento (un año) y devuelve `True` si el año es un año bisiesto, o `False` si no lo es.

La semilla de la función ya se muestra en el código esqueleto del editor.

Nota: también hemos preparado un breve código de prueba, que puedes utilizar para probar tu función.

El código utiliza dos listas - una con los datos de prueba y la otra con los resultados esperados. El código te dirá si alguno de tus resultados no es válido.



Lab 18

4.3.7 LAB Números primos - cómo encontrarlos

Un número natural es primo si es mayor que 1 y no tiene divisores más que 1 y él mismo.

¿Comprobado? De ninguna manera. Por ejemplo, 8 no es un número primo, ya que puedes dividirlo entre 2 y 4 (no podemos usar divisores iguales a 1 y 8, ya que la definición lo prohíbe).

Por otra parte, 7 es un número primo, ya que no podemos encontrar ningún divisor para él.

Tu tarea es escribir una función que verifique si un número es primo o no.

La función:

- se llama `is_prime`.
- toma un argumento (el valor a verificar).
- devuelve `True` si el argumento es un número primo, y `False` de lo contrario.

Sugerencia: intenta dividir el argumento por todos los valores sucesivos (comenzando desde 2) y verifica el resto - si es cero, tu número no puede ser un número primo; analiza cuidadosamente cuándo deberías detener el proceso.

Si necesitas conocer la raíz cuadrada de cualquier valor, puedes utilizar el operador `math.sqrt`. Recuerda: la raíz cuadrada de x es lo mismo que $x^{1/2}$.

Complementa el código en el editor.

Ejecuta la código y verifica si la salida es la misma que la muestra.

Salida esperada:

```
2 3 5 7 11 13 17 19
```

The screenshot shows an IDE with a file explorer on the left containing a folder named 'FUNDAMENTOS1' with subfiles 'Lab1.py' through 'Lab17.py'. The main editor displays the code for 'Lab17.py', which includes a function to check if a number is prime and a loop to print primes up to 30. The terminal at the bottom shows the command to run the script and its output: '2003 -OK', '2011 -OK', '2017 -OK', and '2019 -OK'.

Lab 19

The screenshot shows the same IDE with 'Lab19.py' selected in the file explorer and editor. The code defines two functions: 'liters_100km_to_miles_gallon' and 'miles_gallon_to_liters_100km', along with test cases. The terminal shows the execution of the script, displaying the results of the conversion functions for various inputs.